

Cloud SOC Lab

Detection and Automated Response with Microsoft Sentinel

Prepared by: Izaan Shumaiz

Environment: Microsoft Azure, Microsoft Sentinel

1. Overview

This project involved building a small, cloud-based Security Operations Center (SOC) in Microsoft Azure to gain hands-on experience with Microsoft Sentinel. It started as a honeypot lab: a Windows virtual machine deliberately exposed to the public internet to attract real attack traffic. The project was then extended into a complete detection-to-response pipeline: collecting logs, writing a custom detection rule, generating incidents, and automatically blocking malicious IP addresses through an Azure Logic App. A Sentinel workbook was built at the end to summarize the results.

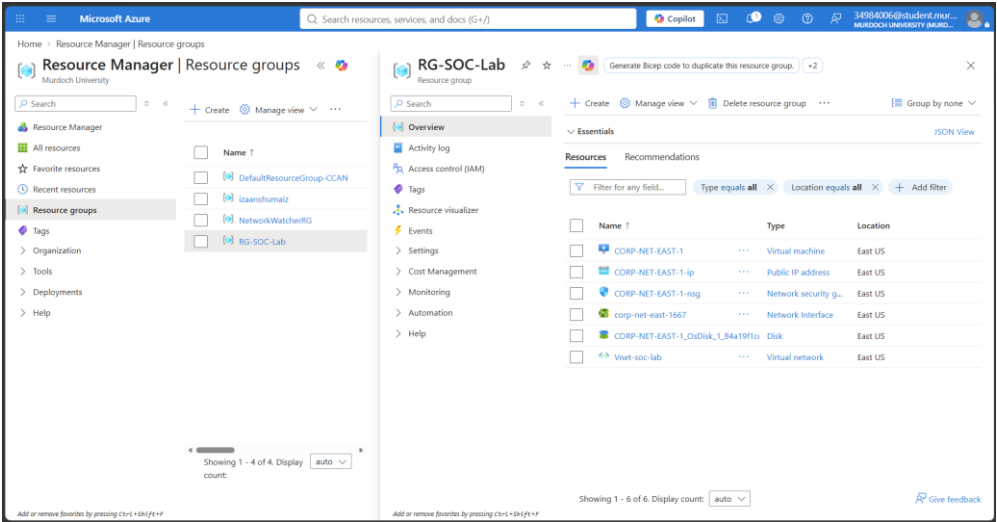
2. Objectives

- Deploy a Windows VM in Azure and expose it to the internet as a honeypot
- Collect Windows security logs using Microsoft Sentinel
- Investigate failed login activity using KQL (Kusto Query Language)
- Build a detection rule to flag repeated failed logins from the same source
- Automatically respond to detected attacks by blocking the source IP
- Visualize the results in a Sentinel workbook

3. Environment

The lab was built entirely in Azure, using the following resources:

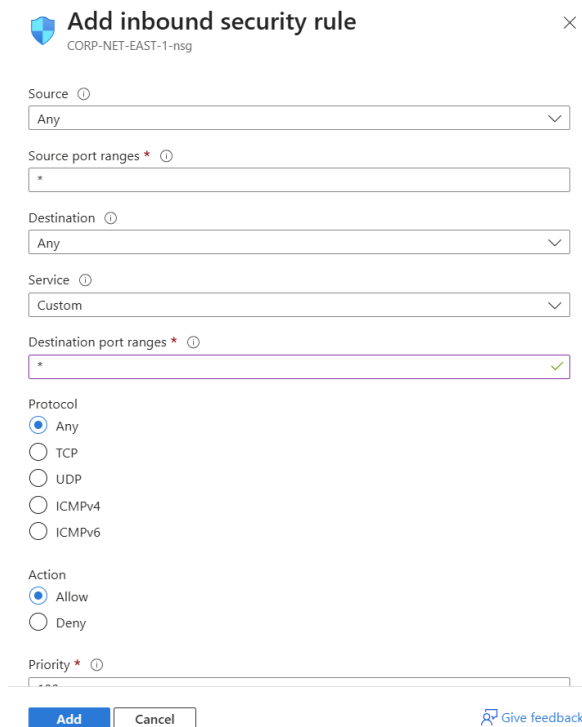
Resource	Name
Resource Group	RG-SOC-Lab
Virtual Machine (honeypot)	CORP-NET-EAST-1
Log Analytics Workspace	LAW-soc-lab-0000
Network Security Group	CORP-NET-EAST-1-nsg
SIEM	Microsoft Sentinel
Automation (Logic App)	SOC-AutoBlock-Malicious-IP



The CORP-NET-EAST-1 virtual machine deployed in Azure

4. Building the Honeypot

The VM was given an ordinary, business-sounding name so it would not stand out as a trap. To make it discoverable, the Network Security Group was opened to allow all inbound traffic, and the Windows Firewall on the VM itself was turned off. A ping test from a separate device confirmed the VM was reachable from the public internet.



Add inbound security rule
CORP-NET-EAST-1-nsg

Source ⓘ
Any

Source port ranges * ⓘ
*

Destination ⓘ
Any

Service ⓘ
Custom

Destination port ranges * ⓘ
*

Protocol
☒ Any
☐ TCP
☐ UDP
☐ ICMPv4
☐ ICMPv6

Action
☒ Allow
☐ Deny

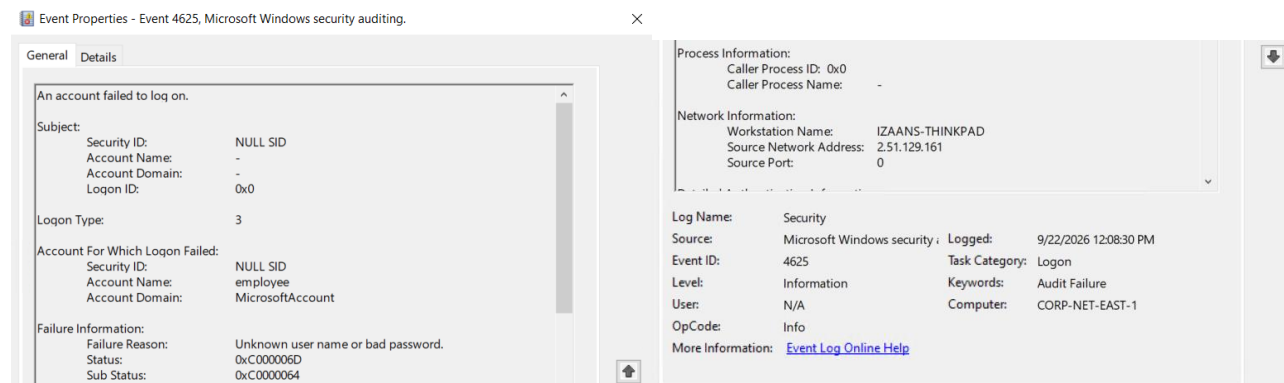
Priority * ⓘ
100

Add Cancel [Give feedback](#)

Inbound rule opened to allow all traffic to the VM

5. Detecting the First Failed Logins

Before building any automation, a few logins were intentionally failed to confirm that Windows was logging the activity correctly. Event Viewer showed these as Event ID 4625 (failed logon), which became the basis for the detection rule built later.



Event Properties - Event 4625, Microsoft Windows security auditing.

General Details

An account failed to log on.

Subject:
Security ID: NULL SID
Account Name: -
Account Domain: -
Logon ID: 0x0

Logon Type: 3

Account For Which Logon Failed:
Security ID: NULL SID
Account Name: employee
Account Domain: MicrosoftAccount

Failure Information:
Failure Reason: Unknown user name or bad password.
Status: 0xC000006D
Sub Status: 0xC0000064

Process Information:
Caller Process ID: 0x0
Caller Process Name: -

Network Information:
Workstation Name: IZAANS-THINKPAD
Source Network Address: 251.129.161
Source Port: 0

Log Name: Security
Source: Microsoft Windows security
Event ID: 4625
Level: Information
User: N/A
OpCode: Info
More Information: [Event Log Online Help](#)

Task Category: Logon
Keywords: Audit Failure
Computer: CORP-NET-EAST-1
Logged: 9/22/2026 12:08:30 PM

Event ID 4625 in Windows Event Viewer, confirming a failed logon is logged

6. Log Collection and Enrichment

A Log Analytics Workspace (LAW-soc-lab-0000) was created and connected to Microsoft Sentinel. The Azure Monitor Agent was configured to forward Windows security events from the VM into this workspace, making them queryable through KQL.

To add geographic context to each failed login, a GeoIP reference list (~55,000 IP ranges) was imported into Sentinel as a watchlist. This allowed each attacking IP address to be resolved to a city and country using a simple lookup in KQL.

Run

Time range : Last 24 hours

Show : 1000 results

```
1 let GeoIPDB_FULL = _GetWatchlist("geoip");
2 let WindowsEvents = SecurityEvent
3     | where IPAddress == "82.114.228.224"
4     | where EventID == 4625
5     | order by TimeGenerated desc
6     | evaluate ipv4_lookup(GeoIPDB_FULL, IPAddress, network);
7 WindowsEvents
8 | project TimeGenerated, Computer, IPAddress, cityname, countryname, latitude, longitude
```

Results

Chart

TimeGenerated [UTC] ↑↓

Computer

IpAddress

cityname

countryname

latitude

longitude

> 9/22/2026, 1:41:53.312 PM

CORP-NET-EAST-1

82.114.228.224

Soisy-sous-Montmorency

France

48.9927

2.3039

> 9/22/2026, 1:41:52.624 PM

CORP-NET-EAST-1

82.114.228.224

Soisy-sous-Montmorency

France

48.9927

2.3039

> 9/22/2026, 1:41:52.092 PM

CORP-NET-EAST-1

82.114.228.224

Soisy-sous-Montmorency

France

48.9927

2.3039

Failed logon events enriched with location data from the GeoIP watchlist

7. Building the Detection Rule

Rather than manually searching for suspicious IPs each time, a repeatable detection rule was built. It flags any source IP responsible for 20 or more failed network logons within a five-minute window:

SecurityEvent
where EventID == 4625
where LogonType == 3
where isempty(IPAddress)
summarize FailedAttempts = count(),
FirstSeen = min(TimeGenerated),
LastSeen = max(TimeGenerated)
by IPAddress, bin(TimeGenerated, 5m)
where FailedAttempts >= 20

This query was converted into a Microsoft Sentinel scheduled analytics rule. The rule was first deployed with a lower threshold (5 failed attempts) to confirm it worked end to end, then raised to 20 once real traffic showed the lower value created far too many incidents for a honeypot receiving thousands of failed logons per hour:

Setting	Value
Name	Multiple Failed Windows Network Logons from Same Source IP
Severity	Medium
MITRE ATT&CK Tactic	Credential Access

Schedule / Lookup period	Every 5 minutes / last 5 minutes
Entity mapping	IpAddress mapped as an IP entity
Incident creation	Enabled

Home > Microsoft Sentinel | Analytics

Analytics rule wizard - Create a new Scheduled rule

Validation passed.

General Set rule logic Incident settings Automated response **Review + create**

Analytics rule details

Name Multiple Failed Windows Logons from Same Source

Description Detects multiple failed Windows authentication attempts from the same source IP and account within a five-minute period.

MITRE ATT&CK > Credential Access (1)

Severity Medium

Status Enabled

Analytics rule settings

Rule query SecurityEvent | where EventID == 4625 | summarize FailedAttempts = count() by IpAddress, Account, bin(TimeGenerated, 5m) | where FailedAttempts >= 5 | order by FailedAttempts desc

Rule frequency Run query every **5 minutes**

Rule period Last **5 minutes** data

Rule start time Automatic

Rule threshold Trigger alert if query returns **more than 0** results

Event grouping Group all events into a single alert

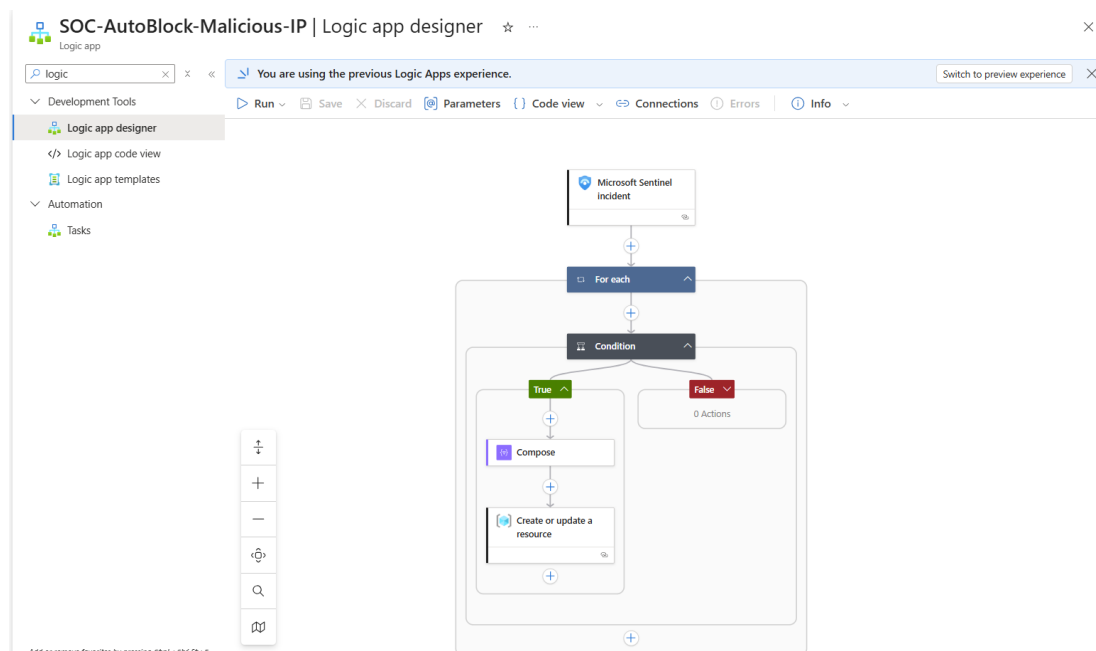
Suppression Not configured

The analytics rule at initial creation, ready for review (threshold later raised to 20)

8. From Incident to Automated Response

Once the rule was active, matching activity generated a Sentinel incident automatically, with the attacking IP attached as an entity. Investigating a few of these incidents manually would not scale, so the next step was to respond automatically.

A Logic App named SOC-AutoBlock-Malicious-IP was built to do this. It is triggered whenever a new Sentinel incident is created, loops through the incident's entities, finds the IP address, and creates a matching deny rule in the VM's Network Security Group.



Logic App workflow: incident trigger, entity loop, condition, and NSG update

Issues Along the Way:

- **Wrong entity path:** the Logic App looked for the entity list at Entities, which returned nothing. Fixed by using the correct path, object -> properties -> relatedEntities, found by inspecting the actual trigger payload.
- **Case-sensitive match:** the condition checked for "ip", but Sentinel sends "Ip". Fixed by matching the exact case.
- **Incorrect nesting:** the condition step was sitting outside the For Each loop instead of inside it. Fixed by moving it inside.
- **Rule priority conflict:** an older "allow any" rule at priority 100 kept letting the traffic through even after the block rule was created. Fixed by lowering that rule to priority 4090, below the automated block.
- **Priority collision:** a second automated block failed because it tried to reuse the same priority number as the first. Left as a documented limitation rather than solved, since it needs dynamic priority handling.
- **Missing permission:** adding a comment to the incident returned a 403 error. Fixed by granting the Logic App's identity a separate Sentinel incident-management permission.

Inbound Security Rules						
4000	SentinelBlock-14-241-68-109	Any	Any	14.241.68.109/32	Any	Deny
4008	SentinelBlock-119-59-117-96	Any	Any	119.59.117.96/32	Any	Deny
4090	DANGER_AllowAnyCusto...	Any	Any	Any	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInBou...	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny

Final rule order: the automated blocks (priority 4000) now takes effect before the old allow-any rule (priority 4090)

9. Verifying the Block

The automation was tested against a real attacking IP, 14.241.68.109, which had been sending a failed login roughly every 9 seconds using a long list of usernames. The last failed attempt was recorded at 8:04 PM, and the NSG block rule was created about a minute later, at 8:05 PM. No further attempts from that IP were logged afterward, confirming the block worked.

Run

Time range: Last 24 hours

Show: 1000 results

KQL mode

```
1 SecurityEvent
2 | where IpAddress == "14.241.68.109"
3 | where EventID == 4625
4 | project TimeGenerated, IpAddress, TargetUserName, LogonType, Computer
5 | order by TimeGenerated desc
6 | take 20
```

Results

Chart

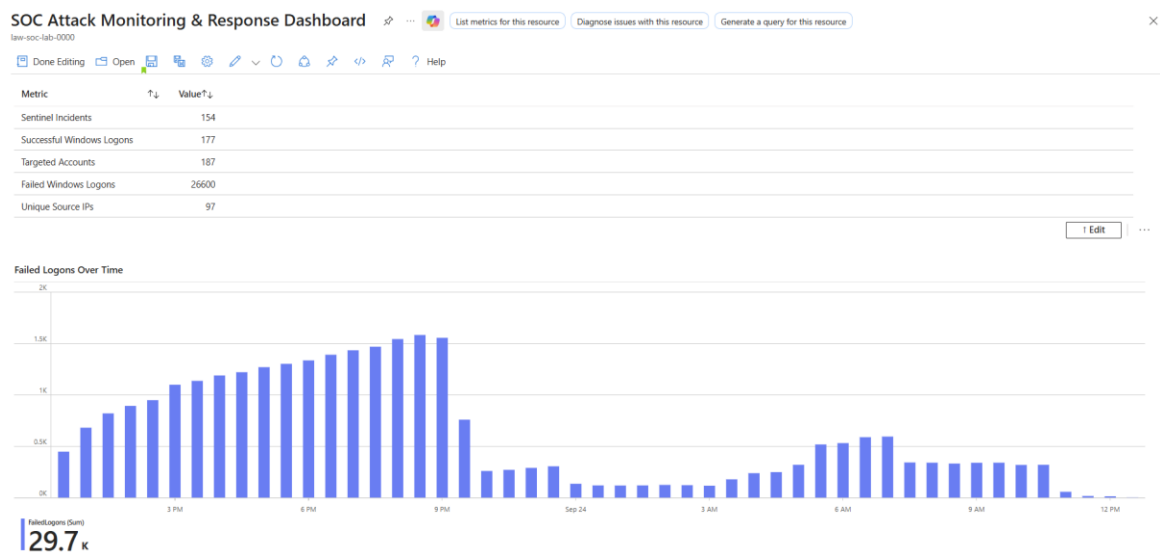
TimeGenerated [UTC]	IpAddress	TargetUserName	LogonType	Computer
> 9/23/2026, 8:04:03.527 PM	14.241.68.109	FTP	3	CORP-NET-EAST-1
> 9/23/2026, 8:03:54.623 PM	14.241.68.109	GUARD	3	CORP-NET-EAST-1
> 9/23/2026, 8:03:45.820 PM	14.241.68.109	ADMIN1	3	CORP-NET-EAST-1
> 9/23/2026, 8:03:37.133 PM	14.241.68.109	USER01	3	CORP-NET-EAST-1
> 9/23/2026, 8:03:28.522 PM	14.241.68.109	123	3	CORP-NET-EAST-1

Failed login attempts from 14.241.68.109, stopping after the automated block was applied

10. Results

A Sentinel workbook was built to summarize the activity collected over the course of the project:

Metric	Value
Failed Windows logons	26,600
Unique source IPs	97
Accounts targeted	187
Sentinel incidents generated	154
Successful Windows logons	177



SOC Attack Monitoring & Response Dashboard: summary metrics

Top Source IPs by Failed Logons			
IpAddress	↑↓	FailedAttempts↑↓	TargetedAccounts↑↓
80.94.95.83		18703	6
119.59.117.96		3197	5
14.241.68.109		2333	176
103.212.182.194		2128	177
163.223.18.58		686	3
103.236.106.125		619	2
80.66.83.80		526	2
203.202.240.151		435	2
141.145.213.159		412	1
86.92.53.191		69	1

Top attacking source IPs by number of failed attempts

The data showed two different attack styles. Some sources, like 80.94.95.83, sent a very high volume of attempts (over 18,000) at a small set of accounts. Others spread far fewer attempts across many more account names, one source alone tried 176 different usernames, suggesting a username-guessing tool rather than a password-guessing one. The two busiest sources, 80.94.95.83 and 195.201.175.13, were checked directly for a follow-up successful logon, and neither produced one.

11. Limitations

- The automated block rule uses a fixed priority number, so it cannot handle two new attacks needing a rule at the same time.
- Only one VM was monitored, so the detection logic was not tested against a larger environment.
- The response (blocking all traffic from an IP) is intentionally aggressive, which suits a lab but would need refinement for production use.

12. Possible Improvements

- Dynamic priority allocation for NSG rules, so multiple blocks can be created at once
- A detection for a successful login that follows a string of failed attempts
- IP reputation lookups (e.g. AbuseIPDB) added to the enrichment step
- Monitoring more than one endpoint

13. Skills Demonstrated

Area	Skills
SIEM	Microsoft Sentinel, Log Analytics, KQL
Detection Engineering	Threshold-based detections, entity mapping, MITRE ATT&CK
SOAR / Automation	Logic Apps, automation rules, RBAC
Azure Networking	Network Security Groups, rule priority
Investigation	Log analysis, timeline reconstruction, GeoIP enrichment